



Biogeography- Based Optimization (BBO)

Por: Samuel, Mario y Álvaro



Índice

1.- Introducción y conceptos clave

2.- Migración y mutación

3.-Definiciones formales

4.-Algoritmo

5.- Diferencias con otros algoritmos

6.- Implementación

7.- Resultados

8.- Conclusiones y referencias

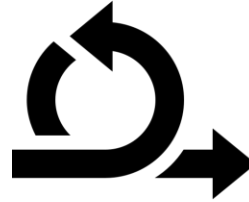
Introducción



Metaheurística inspirada
en la **biogeografía**
(distribución de
especies).



Propuesta por **Dan
Simon (2008)**.



Población fija; mejora
iterativa con **migración**
y **mutación**.



Una solución $\approx$ **hábitat**;
su calidad $\approx$ **HSI (Habitat
Suitability Index)**.

Conceptos Clave



- **Hábitat:** vector de **SIV** (Suitability Index Variables).



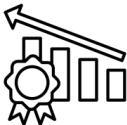
- **HSI:** fitness del hábitat.



- **Ecosistema:** conjunto de hábitats (población).



- **λ (inmigración) y μ (emigración):** dependen del HSI ($\lambda \downarrow$ si $HSI \uparrow$; $\mu \uparrow$ si $HSI \uparrow$).



- **Elitismo:** preserva las mejores soluciones.

Operadores BBO



Migración: comparte SIV entre hábitats según λ y μ .



Mutación: cambios aleatorios controlados por la **probabilidad de especies**.

Ambos operan sobre población **constante**; sin reproducción/descendencia.

En las siguientes diapositivas se explicarán estos operadores en profundidad

Migración

La migración sucede dependiendo de la calidad de la solución (HSI)



Hábitats Buenos (HSI Alto)

Se conceptualizan como hábitats con *muchas especies*

Tienen una **alta tasa de emigración** μ
(Donarán sus buenas características)

Tienen una **baja tasa de inmigración** λ
(No reciben características de sol. peores)



Hábitats Malos (HSI Bajo)

Se conceptualizan como hábitats con *pocas especies*

Tienen una **baja tasa de emigración** μ
(Pocas características buenas que compartir)

Tienen una **alta tasa de inmigración** λ
(No reciben características de sol. peores)

Mecanismo de la migración

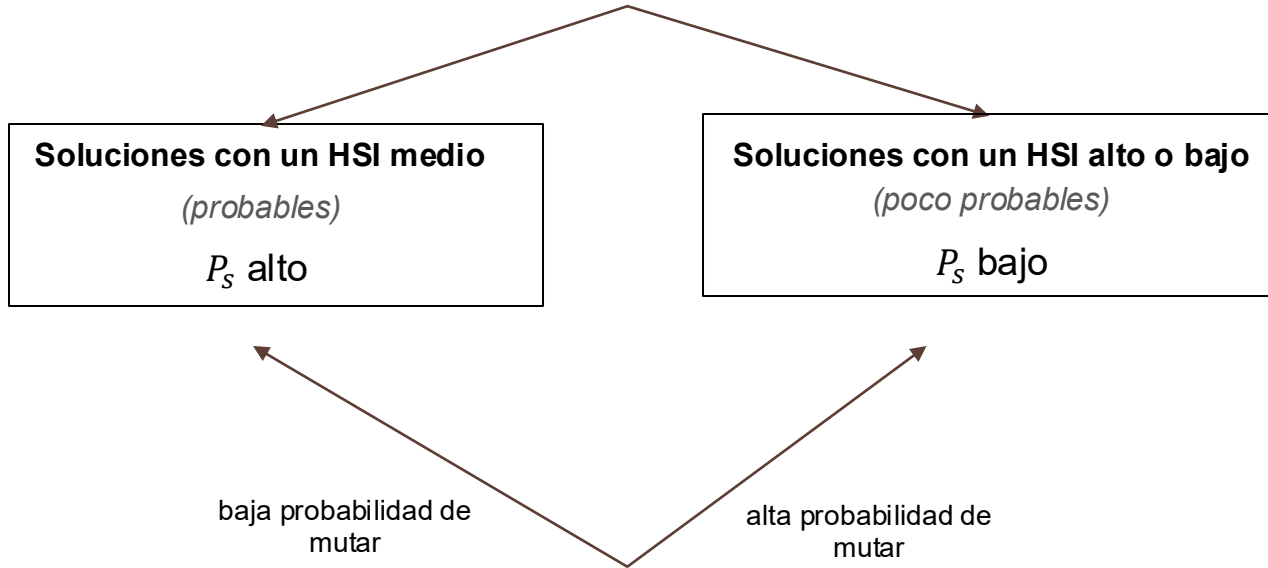
- ① Selección del Receptor H_i para ser modificado, su probabilidad es proporcional a su tasa de inmigración λ_i .
- ② Selección del Donante H_j para donar información, su probabilidad es proporcional a su tasa de emigración μ_j .
- ③ Transferencia de SIV: Se toma un gen (SIV) *aleatorio* de H_j para reemplazar un gen (SIV) aleatorio de H_i .
- ④ Se recalcula el HSI (la calidad) del hábitat H_i modificado.
└─ Se aplica **elitismo**, las mejores soluciones se protegen hacia la siguiente generación

Mutación

La mutación introduce diversidad y modela eventos aleatorios.

①

Probabilidad de Especies P_s : La tasa de mutación depende de la probabilidad de existencia de la especie



②

Tasa de mutación $m(S)$: inversamente proporcional a P_s

Tasa de mutación

La tasa de mutación $m(S)$ se calcula para cada hábitat usando la fórmula:

$$m(S) = m_{\max} \left(\frac{1 - P_s}{P_{\max}} \right)$$

$m_{\max}$ Mutación máxima (*definido por el usuario*)

P_s Probabilidad de esa solución.

$P_{\max}$ Probabilidad máxima encontrada en la población

En la implementación práctica, el *operador* de mutación depende del problema

También se utiliza **elitismo**. Las mejores soluciones se guardan antes de la mutación

Definiciones formales (1/3)

- **Hábitat (H)**: vector de m enteros.
- **HSI**: medida de la bondad de la solución representada por el hábitat.
- **Ecosistema (H^n)**: grupo de n hábitats.
- **Tasa de inmigración, $\lambda(H)$** : monótona **no creciente** con HSI.
- **Tasa de emigración, $\mu(H)$** : monótona **no decreciente** con HSI.
- Usualmente **lineales** y con mismos máximos.



Definiciones formales (2/3)

- **Modificación del hábitat** (Ω): Aplica migración con probabilidad basada en λ y μ .

$$\Omega : H^n \rightarrow H$$

- **Mutación (M)**: operador probabilístico aleatorio sobre H .

$$M : H \rightarrow H$$

- **Transición del ecosistema**

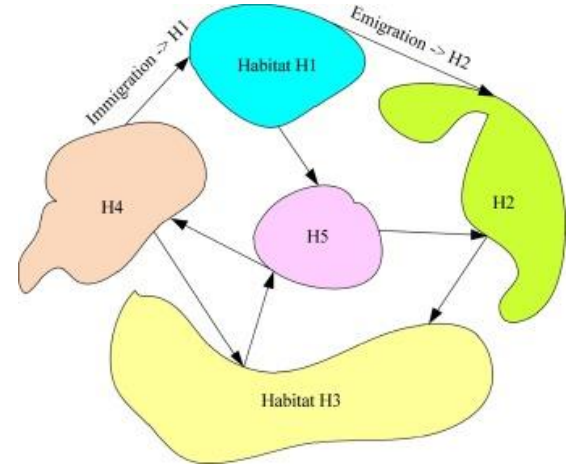
$$\psi = (m, n, \lambda, \mu, \Omega, M) : H^n \rightarrow H^n$$



Definiciones formales (3/3)

Algoritmo BBO ($\mathcal{I}, \psi, \mathcal{T}$) :

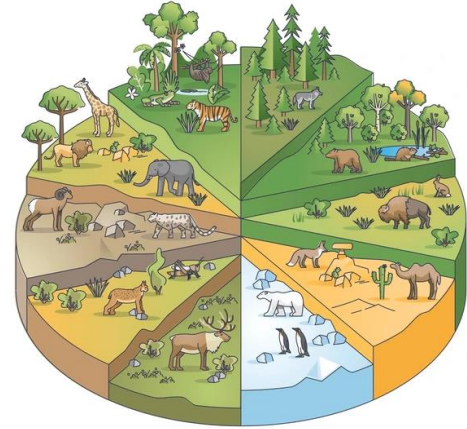
- $\mathcal{I}$, inicializa ecosistema + HSI. $\mathcal{I} : \emptyset \rightarrow H^n, HSI^n$
- ψ , función de transición.
- $\mathcal{T}$, criterio de terminación. $\mathcal{T} : H^n \rightarrow true, false$



Algoritmo

- 1 Inicializar parámetros del BBO.
- 2 Generamos un conjunto aleatorio de hábitats.
- 3 Obtenemos cada HSI y mapeamos:

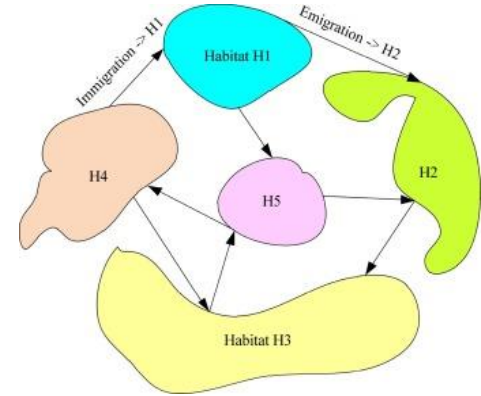
- Número de especies.
- Tasas de inmigración.
- Tasa de emigración.



Algoritmo

④ Utilizamos la migración + reevaluamos HSI:

- Soluciones buenas $\rightarrow$ emigran
- Soluciones malas $\rightarrow$ inmigran



⑤ Actualizamos la probabilidad de las especies de cada hábitat.



Mutación.



⑥ Volvemos al paso (3).

$$\dot{P}_s = \begin{cases} -(\lambda_s + \mu_s)P_s + \mu_{s+1}P_{s+1}, & \text{si } s = 0, \\ -(\lambda_s + \mu_s)P_s + \lambda_{s-1}P_{s-1} + \mu_{s+1}P_{s+1}, & \text{si } 1 \leq s \leq S_{\text{máx}} - 1, \\ -(\lambda_s + \mu_s)P_s + \lambda_{s-1}P_{s-1}, & \text{si } s = S_{\text{máx}}. \end{cases}$$

Diferencias con otros algoritmos

- Sin reproducción ni descendencia (a diferencia de **GA/Estrategias evolutivas**).
- A diferencia de **ACO**, **no** se crea una población nueva cada iteración; se **mantiene** y **adapta** por migración.
- Similar a **PSO/DE** por población fija, pero BBO **intercambia atributos** directamente (SIV) entre soluciones.



Implementación

Entorno



Python 3



Jupyter Notebook



Librerías

- Numpy
- Matplotlib
- Random
- copy

Clase Habitat

Representa una Solución

Guarda la Solución $sivs$ y su Calidad hsi

Almacena las tasas de migración μ y mutación λ

Implementación II

Funciones de test *Zitzler–Deb–Thiele* (2000)

T1: Frente Convexo

```
def test_T1(x):  
    m = len(x)  
    f1 = x[0]  
    g = 1.0 + 9.0 * np.sum(x[1:]) / (m - 1)  
    h = 1.0 - np.sqrt(f1 / g)  
    f2 = g * h  
    return f1, f2
```

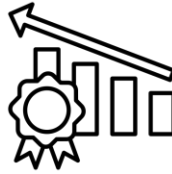
float (T1, T2, T3, T4, T6) y **bits** (T5)

Implementación III

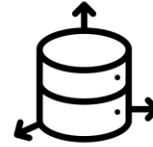
Algoritmo



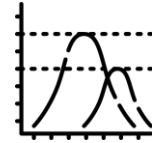
Adaptación **SOEA**
para suma
ponderada



Ordenación por HSI
y Elitismo



Migración
vectorizada



Mutación
dependiente de prob.
de especies. P_s

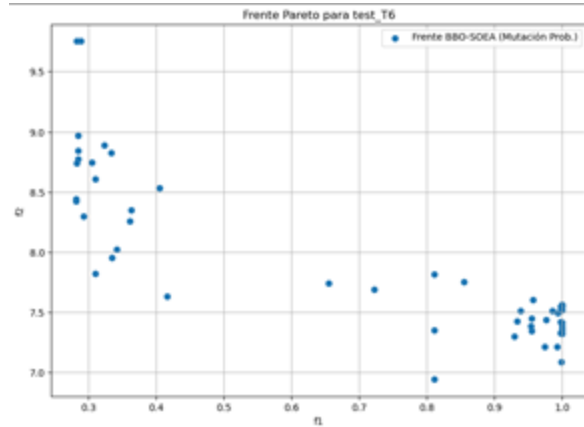
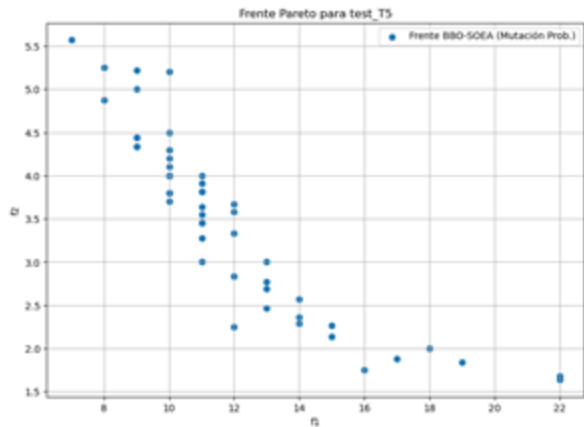
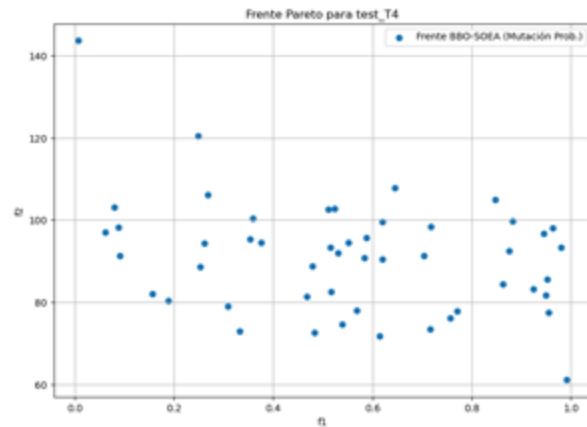
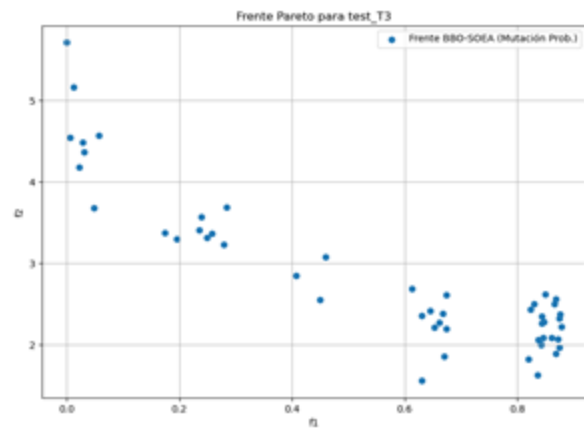
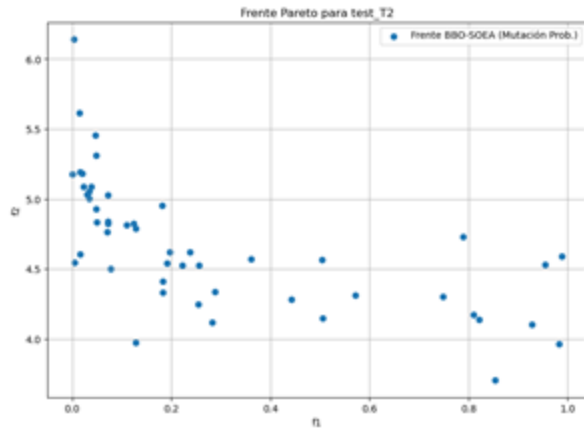
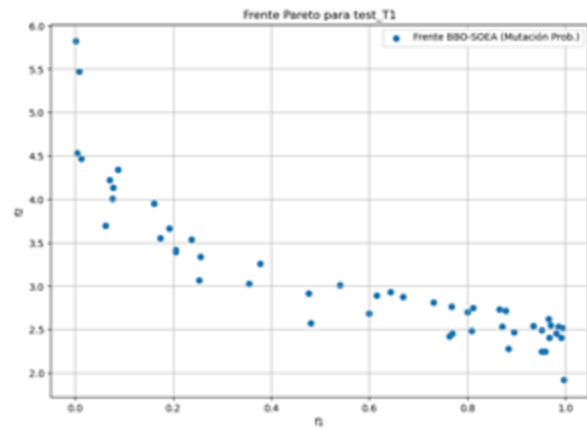
Implementación IV

Simulaciones

```
POP_SIZE = 50  
GENERATIONS = 100  
ELITISM_PARAM = 2  
MAX_MIGRATION = 1.0  
M_MAX = 0.05
```

*Todas las simulaciones se han realizado
con los mismos parámetros*

Resultados



Conclusiones



f1

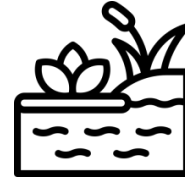


f2

Se ha adaptado BBO de un problema mono-objetivo a multiobjetivo (SOEA)



BBO no funciona bien en todos los problemas de test



Se ha logrado comprender la relación entre la biología y el algoritmo



Referencias

[1] Dan Simon. "Biogeography-based optimization". En: IEEE transactions on evolutionary computation 12.6 (2008), p'ags. 702-713.

[2] Eckart Zitzler, Kalyanmoy Deb y Lothar Thiele. "Comparison of multiobjective evolutionary algorithms: Empirical results". En: Evolutionary computation 8.2 (2000), p'ags. 173-195.